

<< UNIVERSITY NAME >>

<< UNIVERSITY INSTITUTE / FACULTY >>

P.O. Box 0000 - Cameroon | Tel: (237) 000 00 00 00 | www.example.com

School of Bachelors

DEPARTMENT OF COMPUTER ENGINEERING

[INSTITUTION LOGO]

**TOPIC: DESIGN AND IMPLEMENTATION OF A
STUDENT MANAGEMENT SYSTEM**

Case study: SchuLyf

OPTION: SOFTWARE ENGINEERING

Written and Presented by:

<< AUTHOR / GROUP NAME >>

Matricule: << 00000 >>

Under the Supervision of:

<< MR./DR. SUPERVISOR NAME >>

<< Supervisor title >>

2025 - 2026 Academic Year

LIST OF TABLES

Table 1: Historical evolution of student management systems	4
Table 2: Overview of existing student management systems.....	5
Table 3: Comparison of the proposed solutions	9
Table 4: User roles and principal abilities	11
Table 5: Non-functional requirements.....	13
Table 6: Technology stack.....	16
Table 7: Principal database entities by bounded context.....	25
Table 8: Cost of human resources	26
Table 9: Cost of hardware resources	26
Table 10: Cost of software resources	26
Table 11: Total cost estimation	26

LIST OF FIGURES

Figure 1: Layered system architecture of SchuLyf	15
Figure 2: Use case diagram of the Student Management System	17
Figure 3: Class diagram of the principal domain entities	18
Figure 4: Sequence diagram — SAO admission decision.....	19
Figure 5: Sequence diagram — payment validation and receipt issuance	19
Figure 6: Sequence diagram — public receipt verification (HMAC)	20
Figure 7: Activity diagram — online application submission.....	21
Figure 8: Activity diagram — payment standing and exam-gating	23
Figure 9: State diagram — application lifecycle	24
Figure 10: Entity-relationship diagram of the core database.....	25
Figure 11: Login page.....	28
Figure 12: Applicant application form	29
Figure 13: SAO application review queue	30
Figure 14: Accountant payment review with the inline bank-slip viewer.....	31
Figure 15: Student payments screen and payment standing.....	32
Figure 16: Printable, QR-bearing school receipt	33
Figure 17: Public receipt verification page.....	34
Figure 18: Administrator dashboard.....	35
Figure 19: Immutable audit-log modal with filters	36

TABLE OF CONTENTS

LIST OF TABLES.....	ii
LIST OF FIGURES	iii
TABLE OF CONTENTS	iv
GENERAL INTRODUCTION	1
STRUCTURE OF THE WORK IN PARTS	3
CHAPTER 1: STATE OF THE ART OF STUDENT MANAGEMENT SYSTEMS	4
1.1 Introduction	4
1.2 Historical Background and Evolution of Student Management Systems.....	4
1.3 Existing Student Management Systems	5
1.4 Conceptualization of the Proposed System	6
a. Role-Based Access Control	6
b. Digitised Admissions.....	6
c. Tamper-Proof Payments and Receipts.....	6
d. Exam-Gating and Deferrals	6
e. Course Management and Communication.....	7
f. Accountability through an Immutable Audit Trail.....	7
CHAPTER 2: SYSTEM ANALYSIS AND DESIGN	8
PART ONE: PRELIMINARY STUDY	8
2.1 Study of the Existing System.....	8
2.1.2 Limitations of the Manual System.....	8
2.1.3 Proposed Solutions	9
Solution 1: Improved Manual System	9
Solution 2: Adopt a Commercial / Off-the-Shelf SIS.....	9
Solution 3: Custom Student Management System (the chosen solution).....	9

2.1.4 Justification of the Chosen System.....	10
PART TWO: SYSTEM REQUIREMENTS	10
2.2 System Requirements	10
2.2.1 Functional Requirements.....	10
A. Authentication, Authorization and User Management	11
B. Admissions.....	11
C. Payments and Tamper-Proof Receipts.....	12
D. Exam-Gating and Tuition Deferrals	12
E. Course Management.....	12
F. Notifications	13
G. Audit and Reporting	13
2.2.2 Non-Functional Requirements.....	13
A. Security	13
B. Performance	14
C. Reliability and Data Integrity.....	14
D. Usability, Scalability and Maintainability	14
PART THREE: SYSTEM DESIGN, RESOURCES AND COST ESTIMATION	14
2.3 System Architecture	14
2.4 System Design (UML Diagrams).....	16
2.4.1 Use Case Diagram	16
2.4.2 Class Diagram.....	17
2.4.3 Sequence Diagrams	18
2.4.4 Activity Diagrams.....	20
2.4.5 State Diagram	24
2.4.6 Database Design (Entity-Relationship Model).....	24

2.5 Needed Resources and Cost Estimation	25
2.5.1 Needed Resources.....	25
2.5.2 Cost Estimation.....	26
CHAPTER 3: RESULTS AND DISCUSSION	28
3.1 Application Interface Screenshots	28
3.1.1 Login Page	28
3.1.2 Applicant — Application Form	29
3.1.3 Student Affairs Officer — Review Queue	29
3.1.4 Accountant — Payment Review.....	30
3.1.5 Student — Payments and Printable Receipt	31
3.1.6 Public Receipt Verification	33
3.1.7 Administrator — Dashboard and Audit Log	34
3.2 Testing and Validation.....	36
3.3 Discussion.....	36
GENERAL CONCLUSION.....	38
REFERENCES	39

GENERAL INTRODUCTION

The rapid evolution of Information and Communication Technologies (ICT) has transformed nearly every sector of human activity, and higher education is no exception. Universities the world over are replacing paper registers, hand-carried documents, and word-of-mouth communication with integrated digital platforms that manage the entire student lifecycle — from admission, through tuition payment and course work, to the publication of examination results. Such platforms are known generically as **Student Management Systems (SMS)** or Student Information Systems.

In many Cameroonian institutions, however, several core administrative processes remain manual, fragmented, and error-prone. Admission is conducted in person: a candidate fills a physical application form, attaches supporting documents, and deposits the folder at the student-affairs office, then waits to be notified of a decision by mail. Tuition payment is validated by carrying a bank-deposit slip to the accounts office, where it is exchanged for a paper **school receipt** that is easily lost, forged, or shared. Access to examination halls during the exam period depends on a student's payment standing, which is computed by hand. Notices of a lecturer's absence travel informally and are often delivered late or distrusted. These weaknesses are a recurring source of friction, wasted time, and disputes between students and administrators.

This project addresses those weaknesses through the design and implementation of **SchuLyf**, a web-based Student Management System tailored to the realities of a Cameroonian university. SchuLyf digitises five interlocking problem areas: online **admissions**, tamper-proof **payments and receipts**, **exam-gating and tuition deferrals**, **course management**, and **notifications** — all underpinned by role-based access control and an immutable audit trail that records every significant action.

The system was developed using the **Laravel 13** PHP framework with an **Inertia.js + Vue 3** single-page front end, a **MySQL** database, and modern software-engineering practices including UML-based modelling, an action-oriented service layer, automated testing with Pest, and a security posture hardened against a documented audit. The remainder of this report presents the state of the art, the analysis and design of the system, and the results obtained.

STRUCTURE OF THE WORK IN PARTS

This work is organised into three chapters, the second of which is divided into three parts. The structure is as follows:

- **Chapter 1 — State of the Art of Student Management Systems:** the history and evolution of student management, a survey of existing systems, and the conceptualization of the proposed system.
- **Chapter 2 — System Analysis and Design,** divided into three parts:
 - **Part One — Preliminary Study:** study of the existing (manual) system, its limitations, the proposed solutions, and the justification of the chosen solution.
 - **Part Two — System Requirements:** the functional and non-functional requirements of the new system.
 - **Part Three — System Design:** the architecture and UML models, the database design, and the needed resources and cost estimation.
- **Chapter 3 — Results and Discussion:** the application interface, a discussion of how the implemented features meet the objectives, and the testing performed.

The work closes with a general conclusion and a list of references.

CHAPTER 1: STATE OF THE ART OF STUDENT MANAGEMENT SYSTEMS

1.1 Introduction

Education is one of the most information-intensive activities of any society. A single university must keep accurate, current records of who has applied, who has been admitted, who has paid, who is enrolled in which course, who attended which session, and what marks each student earned. Historically these records were kept on paper and managed by hand. The growth of student populations and the maturity of ICT have made **Student Management Systems** indispensable: they automate admissions, registration, fee management, course administration, results processing, and communication, while improving accuracy, transparency, and accountability.

This chapter reviews the state of the art of student management systems. It traces their historical evolution, surveys representative existing solutions, identifies their strengths and gaps, and sets out the conceptual foundations of the system proposed in this work.

1.2 Historical Background and Evolution of Student Management Systems

Record-keeping in education is as old as formal education itself. For centuries, student records — enrolment lists, fee ledgers, and examination results — were written by hand in registers and stored in filing cabinets. The methods, tools, and limitations evolved alongside the broader history of computing, as summarised in Table 1.

Table 1: Historical evolution of student management systems

Period	Era	Key characteristics	Tools & methods	Limitations
Before 1960	Manual records	Hand-written registers and fee ledgers; in-person admission	Paper registers, filing cabinets	Data loss, no search, slow retrieval
1960s–1980s	Mainframe era	Centralised batch processing of student records	Punch cards, mainframes	High cost, no real-time access, specialist staff
1980s–1990s	Desktop databases	Departmental record-keeping on personal computers	dBASE, FoxPro, spreadsheets	Data silos, single-machine, duplication

1990s–2000s	Client–server SIS	Networked student-information systems on campus LANs	Relational databases, LAN clients	Limited remote access, costly licensing
2000s–2010s	Web-based systems	Online portals and learning-management systems	Web SIS, Moodle, student portals	Integration cost, security concerns
2010s–present	Cloud & mobile	Integrated cloud platforms with analytics and APIs	Cloud SIS, web frameworks, mobile apps	Subscription cost, data-sovereignty concerns
Proposed	SchuLyf	Secure, role-based, tamper-proof receipts, immutable audit	Laravel, Inertia/Vue, MySQL	Requires hosting and internet access

1.3 Existing Student Management Systems

A number of student management and student-information systems are in use today, ranging from open-source learning platforms to enterprise-grade commercial suites. Each digitises part of the student lifecycle, but few are well adapted to the specific administrative realities — and budgets — of a single faculty in a developing-country university. Common limitations include high licensing cost, excessive complexity, a focus on the learning side at the expense of admissions and finance, and the absence of locally relevant features such as bank-slip validation and tamper-proof school receipts. Table 2 compares representative systems.

Table 2: Overview of existing student management systems

System	Type / focus	Strengths	Limitations
Moodle	Learning management system	Open-source, rich course tools	Not a full SIS; no admissions or fee-receipt workflow
Fedena	School/college ERP	All-in-one modules	Generic, heavyweight, paid advanced modules
openSIS	Open-source SIS	Free, student-information focus	K-12 oriented, dated interface, limited finance
Ellucian Banner	Enterprise SIS	Comprehensive, scalable	Very costly, complex, over-featured for a faculty
PowerSchool	Commercial SIS	Mature, well supported	Subscription cost, limited local customization
Local manual system	Paper-based	Low cost, familiar to staff	Data loss, forgeable receipts, no audit trail
Proposed (SchuLyf)	Custom web SMS	Tailored, role-based, tamper-proof receipts,	Requires internet and hosting

		audited	
--	--	---------	--

1.4 Conceptualization of the Proposed System

The conceptualization of SchuLyf translates the day-to-day operations of student administration — admission, payment validation, exam clearance, course delivery, and communication — into a single, secure, role-based digital platform. The design is organised around the following principles:

a. Role-Based Access Control

Six roles — **Applicant, Student, Lecturer, Accountant, Student Affairs Officer (SAO), and Administrator** — each see only the functions appropriate to their responsibilities. This protects sensitive operations (admission decisions, payment validation, results publication, user management) and keeps each user's workspace focused and uncluttered.

b. Digitised Admissions

Applicants apply online against cascading programme and level menus, upload the exact documents their programme requires, and track the outcome. An SAO triages and decides each application; on admission, the applicant atomically becomes a student with a uniquely generated **matricule**, replacing the manual paper-folder process.

c. Tamper-Proof Payments and Receipts

A student reports a bank deposit by uploading the slip, so a lost paper copy is no longer fatal. An accountant validates the deposit, and the system issues a single **HMAC-signed school receipt** that is bound to the student's identity and can be verified by anyone at a public endpoint — making forgery and reuse detectable.

d. Exam-Gating and Deferrals

Tuition is paid in dated installments. The system computes each student's **payment standing** live (cleared, blocked, or deferred) and uses it to gate examination and facility access, while an accountant-granted **deferral** can lift the gate for a defined period.

e. Course Management and Communication

Courses are planned and approved, sessions are scheduled, attendance and assignments are recorded, and continuous-assessment and examination marks are published with a student dispute channel. Lecturer-absence and session changes are pushed to the affected cohort through reliable email and in-app **notifications**.

f. Accountability through an Immutable Audit Trail

Every significant action — a decision, a validation, a role change, a login — is written to an append-only **audit log** that can never be edited or deleted through the application, giving administrators a tamper-evident record for monitoring and dispute resolution.

Together these principles deliver a secure, scalable, and user-friendly platform that overcomes the limitations of manual student administration. This conceptual design is the foundation for the system architecture, database design, UML models, and user interfaces presented in Chapter 2.

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

This chapter presents the analysis and design of the SchuLyf Student Management System. It is divided into three parts: a preliminary study of the existing system and the justification of the chosen solution; the functional and non-functional requirements; and the system design, comprising the architecture, UML models, database design, and resource and cost estimation.

PART ONE: PRELIMINARY STUDY

2.1 Study of the Existing System

The first step in developing the system was to study how student administration is currently carried out in the modelled university. The processes are predominantly manual and paper-based:

- **Admission** — a candidate collects and fills a physical application form, attaches photocopies of credentials, and deposits the folder at the student-affairs office; a decision is reached manually and communicated by mail.
- **Payment validation** — a student deposits tuition into one of the institute's bank accounts (UBA, AFG, AFRICLAND, SCB, SGC), then carries the bank slip to the accounts office where it is checked and exchanged for a paper school receipt.
- **Exam clearance** — before examinations, staff manually verify that each student has paid the installment due; those short of the threshold are denied access unless granted a written deferral.
- **Course information** — class schedules, lecturer-absence notices, attendance, and results are managed on paper or by word of mouth.

2.1.2 Limitations of the Manual System

- High risk of loss or damage to physical application folders and receipts.
- A paper school receipt can be **forged, altered, or shared** between students.
- Students who lose a bank slip cannot prove payment, causing disputes with staff.
- Manual computation of payment standing is slow and inconsistent.
- Absence notices and results travel informally, are delayed, and are distrusted.

- No reliable, tamper-evident record of who did what and when (no audit trail).

These limitations show that the manual system is inefficient, insecure, and difficult to scale, motivating a tailored digital solution.

2.1.3 Proposed Solutions

Three approaches were considered to address the limitations identified above.

Solution 1: Improved Manual System

Standardised forms, pre-printed serial-numbered receipts, and spreadsheet ledgers could reduce some errors. **Advantages:** low cost, familiar to staff. **Limitations:** still prone to loss and forgery, no real-time standing, no audit, and poor scalability.

Solution 2: Adopt a Commercial / Off-the-Shelf SIS

A commercial student-information system or ERP could be purchased. **Advantages:** ready-made, feature-rich. **Limitations:** recurring licensing cost, excessive complexity, weak fit to local workflows such as bank-slip validation, and dependence on a third-party vendor.

Solution 3: Custom Student Management System (the chosen solution)

A custom web application — SchuLyf — built specifically for the faculty's workflows, integrating role-based access, online admissions, tamper-proof receipts, exam-gating, course management, notifications, and an immutable audit log. **Advantages:** tailored, cost-effective in the long term, secure, and extensible. **Limitations:** requires initial development effort and ongoing maintenance.

Table 3: Comparison of the proposed solutions

Solution	Key features	Advantages	Limitations
Improved manual	Standard forms, serial receipts, spreadsheets	Low cost, familiar	Forgery and loss persist, no audit, poor scaling
Commercial SIS	Ready-made modules, broad feature set	Fast to deploy, feature-rich	Costly, complex, weak local fit, vendor lock-in
Custom SMS	Role-based access, HMAC	Tailored, secure, cost-	Development time

(SchuLyf)	receipts, exam-gating, audit	effective, extensible	and maintenance
-----------	------------------------------	-----------------------	-----------------

2.1.4 Justification of the Chosen System

The custom system was chosen for the following reasons:

- **Tailored functionality** — it implements exactly the faculty's processes (bank-slip validation, year-scoped matricules, installment-based exam-gating) rather than forcing them into a generic product.
- **Cost-effectiveness** — it is built on a free, open-source stack and avoids recurring per-seat licensing fees.
- **Security and accountability** — role-based access, hashed credentials, HMAC-signed receipts, and an immutable audit log directly target the trust and fraud problems of the manual process.
- **Scalability and maintainability** — a clean, layered architecture lets new features be added without redesigning the system.
- **Improved communication and data accuracy** — automated notifications and centralised, validated data reduce delays, duplication, and disputes.

PART TWO: SYSTEM REQUIREMENTS

2.2 System Requirements

System requirements define what the SchuLyf Student Management System must do (functional requirements) and the quality attributes under which it must operate (non-functional requirements). They were derived from the study of the existing system, stakeholder needs, and the project objectives.

2.2.1 Functional Requirements

The functional requirements describe the core operations the system performs, grouped by module. Table 3 summarises the six user roles and their principal abilities, after which each module's requirements are detailed.

Table 4: User roles and principal abilities

Role	Principal abilities
Applicant	Register, submit and track an admission application, upload documents
Student	Report payments, view standing, request deferrals, view courses/attendance/assignments/results, raise disputes, print receipts
Lecturer	Manage assigned courses and plans, schedule sessions, mark attendance, set and grade assignments, record marks
Accountant	Validate/reject payments, issue receipts, review deferral requests, look up a student's standing
Student Affairs Officer	Triage and decide applications, manage courses and lecturers, approve plans, publish results, resolve disputes
Administrator	Manage users and reference data, configure fee schedules, view the audit log; may perform every other role's actions

A. Authentication, Authorization and User Management

- Users authenticate with a single login field that accepts an **email, employee ID, phone number, or matricule**, resolved in one query.
- Passwords are stored using strong one-way **hashing**; optional two-factor authentication (TOTP) and email verification are supported.
- Access to every feature is restricted by **role**; sensitive actions are further protected by ability gates and per-resource ownership checks.
- Administrators create staff accounts that receive a single-use **invite link** to set their own password; accounts can be deactivated, restored, and have their role changed.
- Login attempts and password resets are **rate-limited** to resist brute force.

B. Admissions

- Applicants build an application against **cascading dropdowns** (degree programme → department → offering → level) and upload the documents required for that programme and level, in addition to the always-required national ID and birth certificate.
- The system enforces **one open application at a time** per applicant.
- An SAO **triages** an application through reversible interim states and records a single terminal **decision** (admitted, rejected, or waitlisted).

- On admission, the applicant is **promoted to a student** atomically: a student profile and a unique year-scoped matricule are created and the Student role is assigned.
- A returning student's prior enrolment can be **restored** instead of issuing a fresh record.

C. Payments and Tamper-Proof Receipts

- A student **reports a bank deposit** by selecting the bank, entering the amount and reference, and uploading the slip; the document is stored so a lost paper copy is no longer fatal.
- An accountant **validates or rejects** each submission; validation **issues an immutable, HMAC-signed school receipt** bound to the student's identity.
- Anyone may **verify a receipt** at a public endpoint; a forged number, altered amount, or reuse by another student is detected and reported as invalid.
- Administrators configure **fee schedules** and dated **installments** per programme, level, and academic year.

D. Exam-Gating and Tuition Deferrals

- The system computes each student's **payment standing** live — cleared, blocked, or deferred — from the fee schedule, validated payments, and active deferrals.
- A blocked student may **request a deferral**; an accountant approves it with an extended deadline or rejects it with a reason.
- Gate-facing staff can **look up any student's standing by matricule**.

E. Course Management

- An SAO publishes courses and **assigns a lecturer**; the lecturer drafts a **course plan** for SAO approval.
- On an approved plan the lecturer **schedules sessions, marks attendance** for the cohort, and creates and **grades assignments**.
- Continuous-assessment and examination marks are recorded, then **published** by an SAO; the final score and letter grade are computed automatically.
- Students view **published results** and may **raise a dispute**, which an SAO resolves.

F. Notifications

- The system sends **email and in-app notifications** for admission decisions, payment outcomes, deferral decisions, and course-session cancellations or reschedules.
- Session-change notifications are fanned out to the **entire affected cohort**, replacing unreliable word-of-mouth notices.

G. Audit and Reporting

- Every significant write is recorded in an **immutable audit log** with the actor, action, before/after changes, and request context.
- Administrators read the audit log through a filterable modal; role and application dashboards summarise counts and recent activity.

2.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and constraints of the system, summarised in Table 4 and detailed below.

Table 5: Non-functional requirements

Attribute	Requirement
Security	Hashed passwords, role-based access, ability gates, HMAC-signed receipts, hardened file viewers, rate limiting, input validation
Performance	Responsive pages, indexed queries, cached reference data, queued email so requests stay fast
Reliability	Atomic database transactions, terminal-state re-guards, graceful error handling
Usability	Clean, consistent role-based interface; clear forms and feedback toasts
Scalability	Layered architecture; new modules added without redesign; cache and queue back-ends swappable
Maintainability	Action-class pattern, typed routes, automated tests, and documentation
Compatibility	Runs in modern browsers; responsive layout for desktop and mobile
Data integrity	Foreign-key constraints, soft deletes, immutable receipt and audit records

A. Security

Authentication is backed by Laravel Fortify on a session guard. Passwords use strong hashing; in production a 12-character policy with breach checking is enforced. Authorization is layered

— role middleware, ability gates, and per-resource ownership checks. School receipts are HMAC-signed and the inline file viewers apply a strict MIME allowlist and sandboxing content-security policy. Sensitive endpoints are rate-limited.

B. Performance

Reference data (departments, offerings, document types) is served through a read-through cache; database queries are backed by composite indexes; and email and notifications are dispatched on a queue after the database transaction commits, so user requests are never blocked by slow I/O.

C. Reliability and Data Integrity

Every state change runs inside a database transaction with a row lock and a terminal-state re-guard, so a record cannot be decided twice and partial writes roll back cleanly. Receipts and audit records are immutable; most other tables use soft deletes so data is recoverable.

D. Usability, Scalability and Maintainability

The interface is built with the PrimeVue component library on a consistent emerald design system, presenting each role with only the actions relevant to it. The layered, action-oriented architecture and a comprehensive automated test suite make the system straightforward to extend and maintain.

PART THREE: SYSTEM DESIGN, RESOURCES AND COST ESTIMATION

2.3 System Architecture

SchuLyf follows a layered, server-driven single-page-application architecture. The Vue 3 front end communicates with the Laravel back end through Inertia.js: a page navigation is an XHR that returns a JSON page object which the client renders without a full reload. On the server, routes pass through an authentication, email-verification, and role-middleware stack before reaching a thin controller. **Every state change is encapsulated in a single-purpose Action class** that opens a transaction, locks the target row, re-guards its status, writes the

change, records an audit entry, and dispatches any notification after the transaction commits. Figure 1 shows the layered architecture.

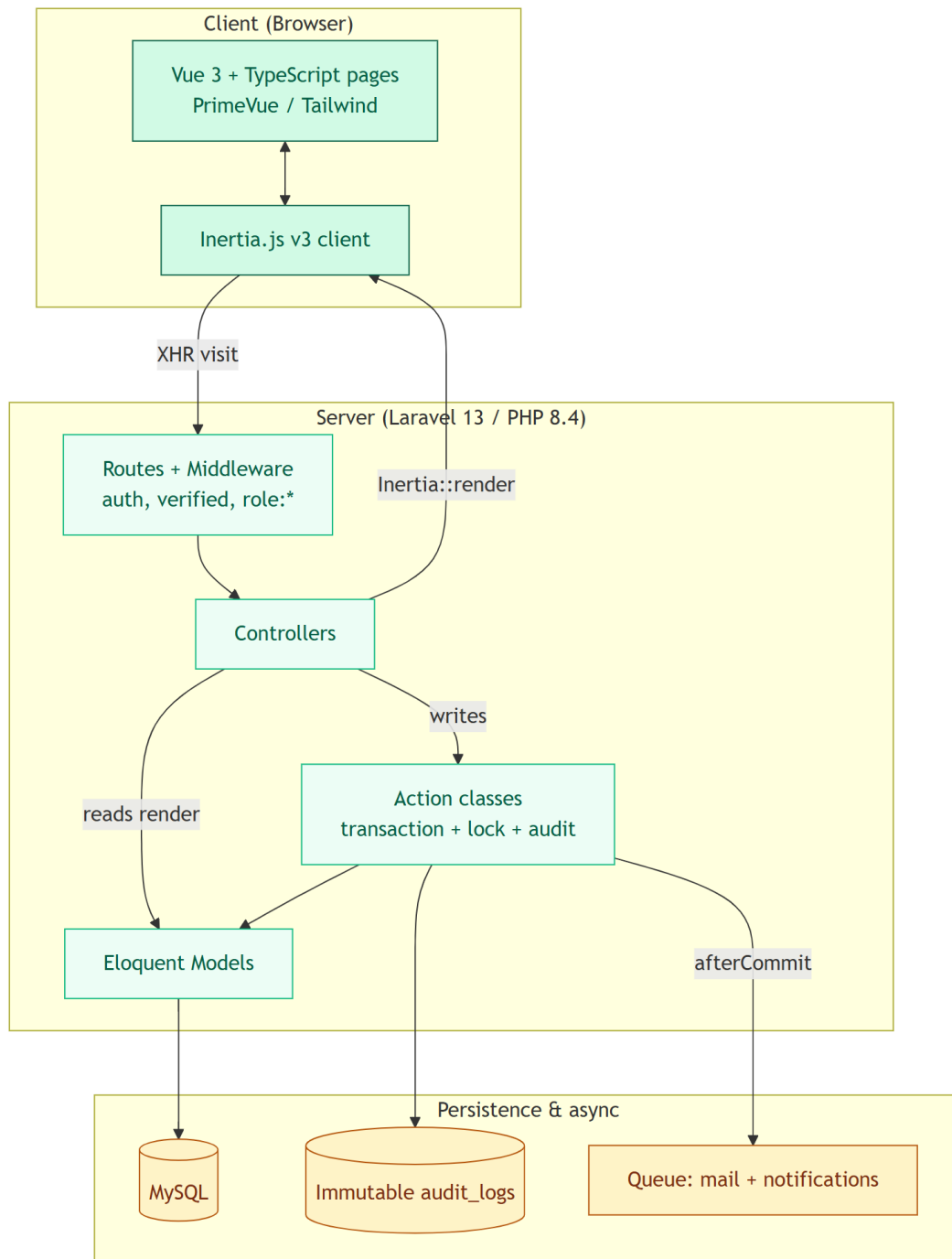


Figure 1: Layered system architecture of SchuLyf

The technology stack is summarised in Table 5. The choice of a mature open-source stack keeps licensing cost at zero while providing strong security defaults, a powerful object-relational mapper, and a modern, reactive user interface.

Table 6: Technology stack

Layer	Technology	Role
Language / framework	PHP 8.4, Laravel 13	Application framework and domain logic
Authentication	Laravel Fortify	Login, registration, reset, email verification, 2FA
SPA bridge	Inertia.js v3	Server-routed single-page application
Front end	Vue 3, TypeScript, Vite	Reactive interface components
UI library	PrimeVue (Aura), Tailwind CSS v4	Forms, tables, dialogs, design system
Database	MySQL	Relational persistence
Cache / queue	File or Redis cache; database queue	Reference cache, queued mail and notifications
Testing	Pest v4, Laravel Pint	Automated tests and code formatting

2.4 System Design (UML Diagrams)

The system was modelled with the Unified Modeling Language (UML) before and during implementation. This section presents the use-case, class, sequence, activity, and state diagrams, together with the entity-relationship model of the database.

2.4.1 Use Case Diagram

The use-case diagram (Figure 2) presents the major actors — Applicant, Student, Lecturer, Accountant, Student Affairs Officer, Administrator, and an anonymous public verifier — and the principal functions each performs within the system. For readability it is shown in two panels: the upper panel covers admissions, payments and exam clearance, and the lower panel covers course management and administration.

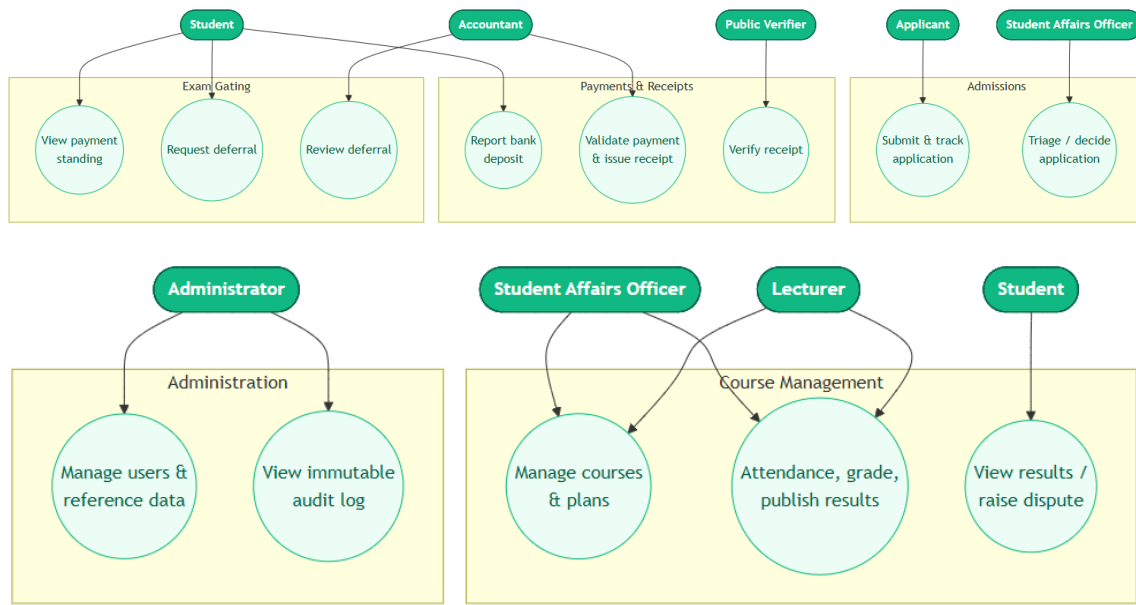


Figure 2: Use case diagram of the Student Management System

2.4.2 Class Diagram

The class diagram (Figure 3) shows the main domain entities, their key attributes and operations, and the relationships between them — for example, a validated payment submission yields exactly one immutable school receipt, and an admitted application gives rise to a student profile.

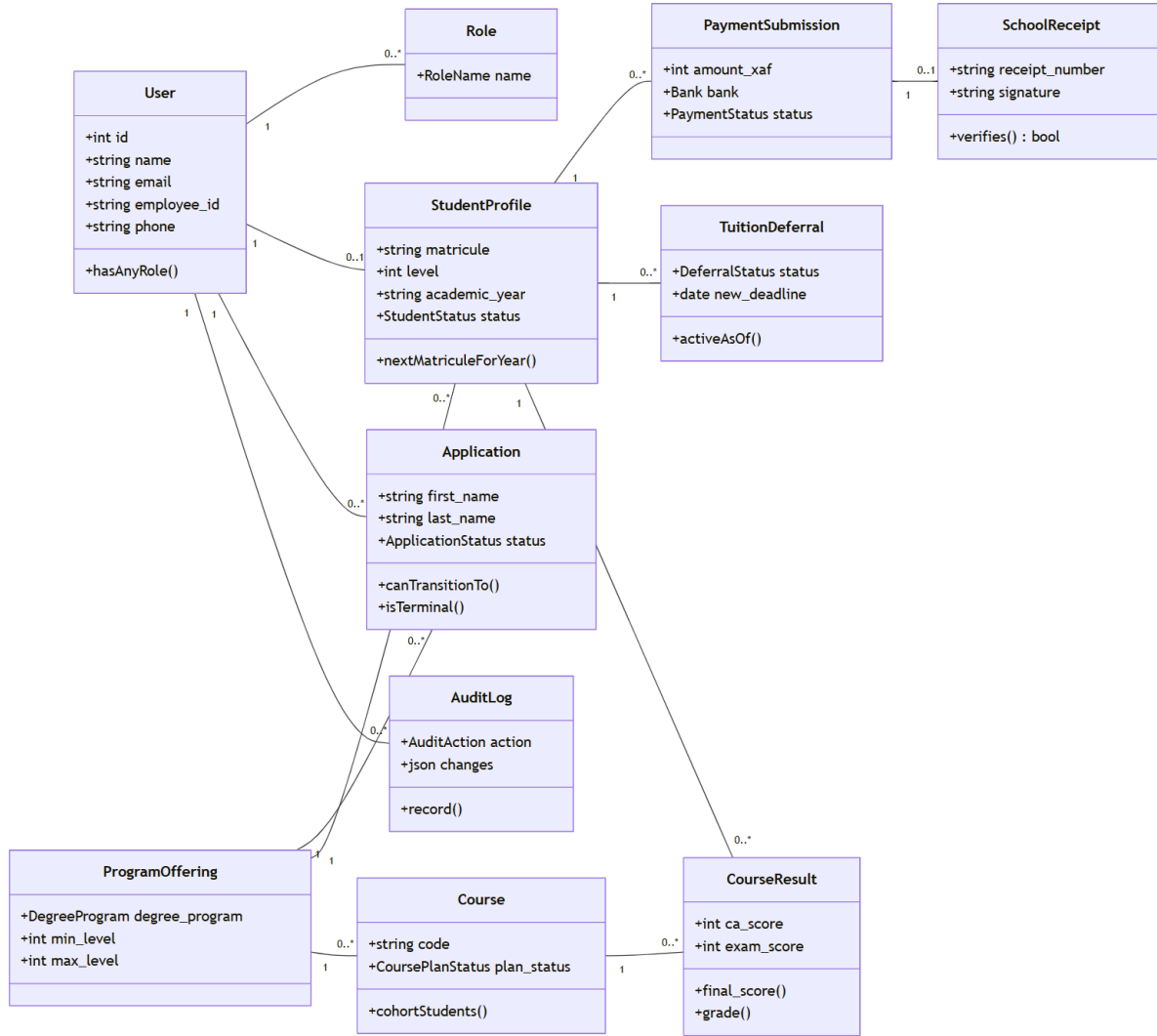


Figure 3: Class diagram of the principal domain entities

2.4.3 Sequence Diagrams

Sequence diagrams illustrate the interactions between objects over time for the system's most important workflows.

Figure 4 shows the **admission decision**: when an SAO admits an application, the decision action re-fetches the record under a lock, updates its status, mints a student profile and matricule, assigns the Student role, writes an audit entry, and — only after the transaction commits — dispatches the decision email.

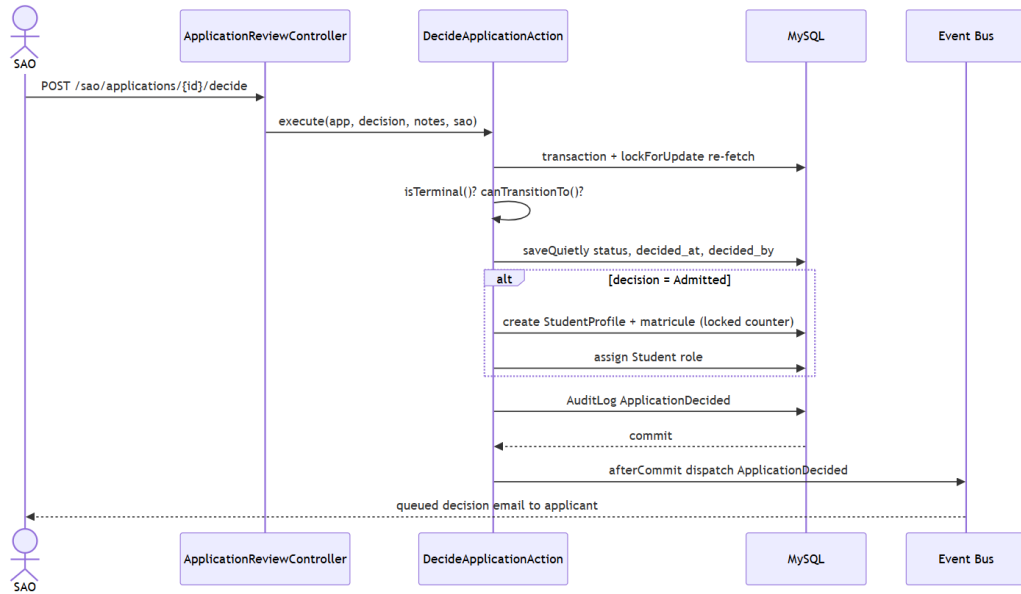


Figure 4: Sequence diagram — SAO admission decision

Figure 5 shows **payment validation and receipt issuance**: the accountant's action locks the submission, re-guards its terminal status, marks it validated, draws the next receipt number from a locked per-year counter, computes the HMAC signature over the receipt number, matricule, amount, and academic year, and creates the immutable receipt.

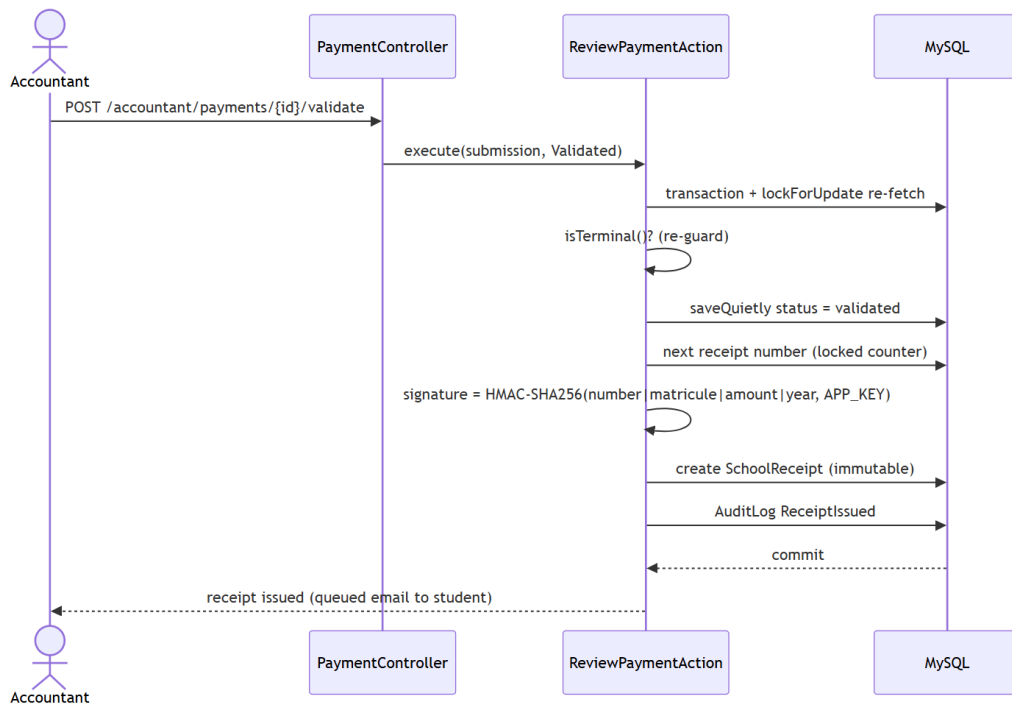


Figure 5: Sequence diagram — payment validation and receipt issuance

Figure 6 shows **public receipt verification**: the verifier supplies only a receipt number, the system recomputes the HMAC from the receipt's currently bound identity, and reveals the bound details only when the signature matches; an unknown, forged, or reused receipt reads simply as invalid.

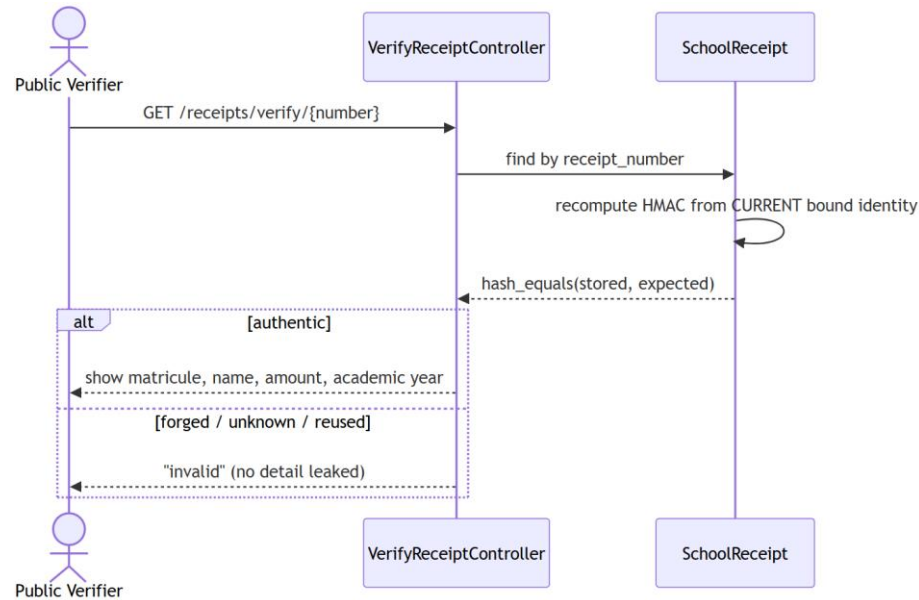


Figure 6: Sequence diagram — public receipt verification (HMAC)

2.4.4 Activity Diagrams

Activity diagrams describe the flow of control through a process. Figure 7 models the **online application** funnel from registration to submission, including the one-open-application guard.

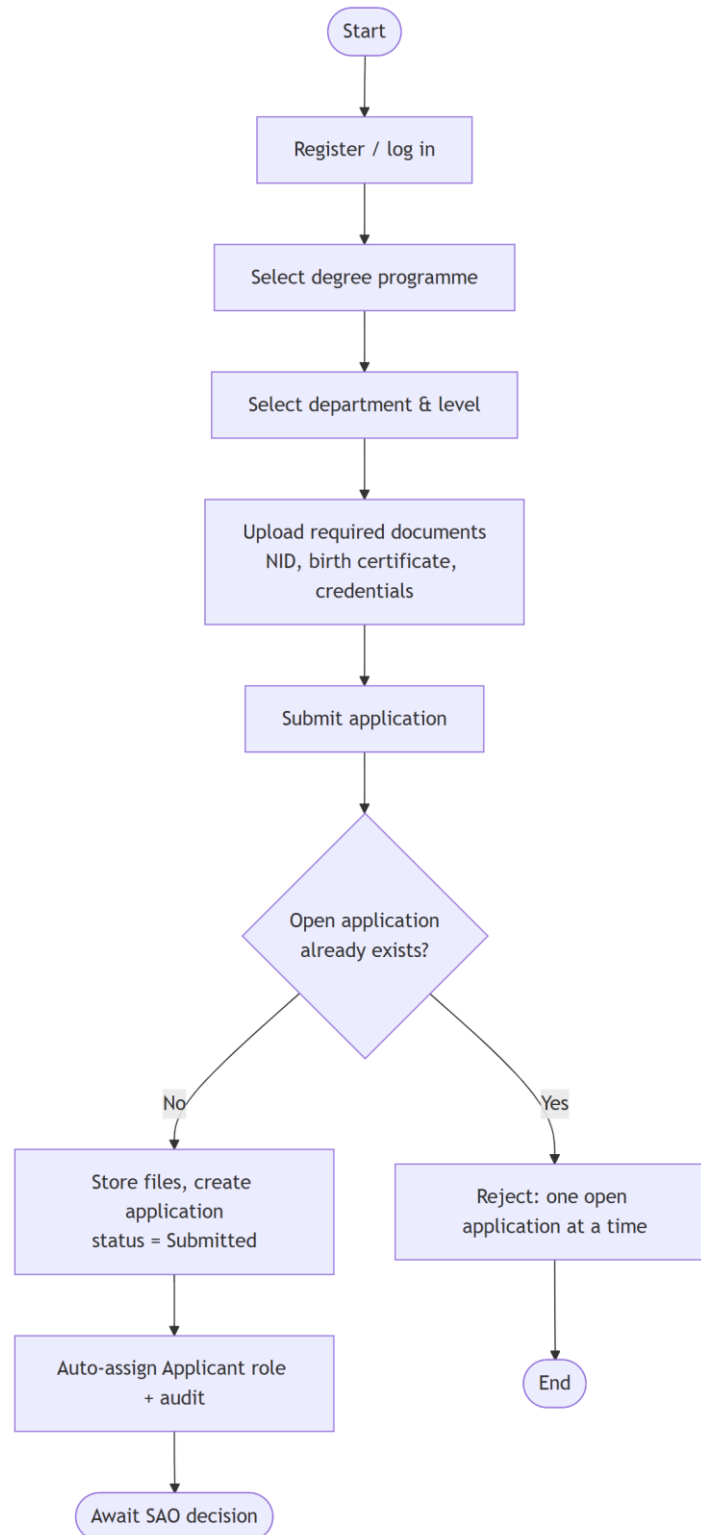


Figure 7: Activity diagram — online application submission

Figure 8 models the **payment-standing computation** that drives exam-gating: the service compares the installments due so far against validated payments and, on a shortfall, checks for an active deferral before returning cleared, deferred, or blocked.

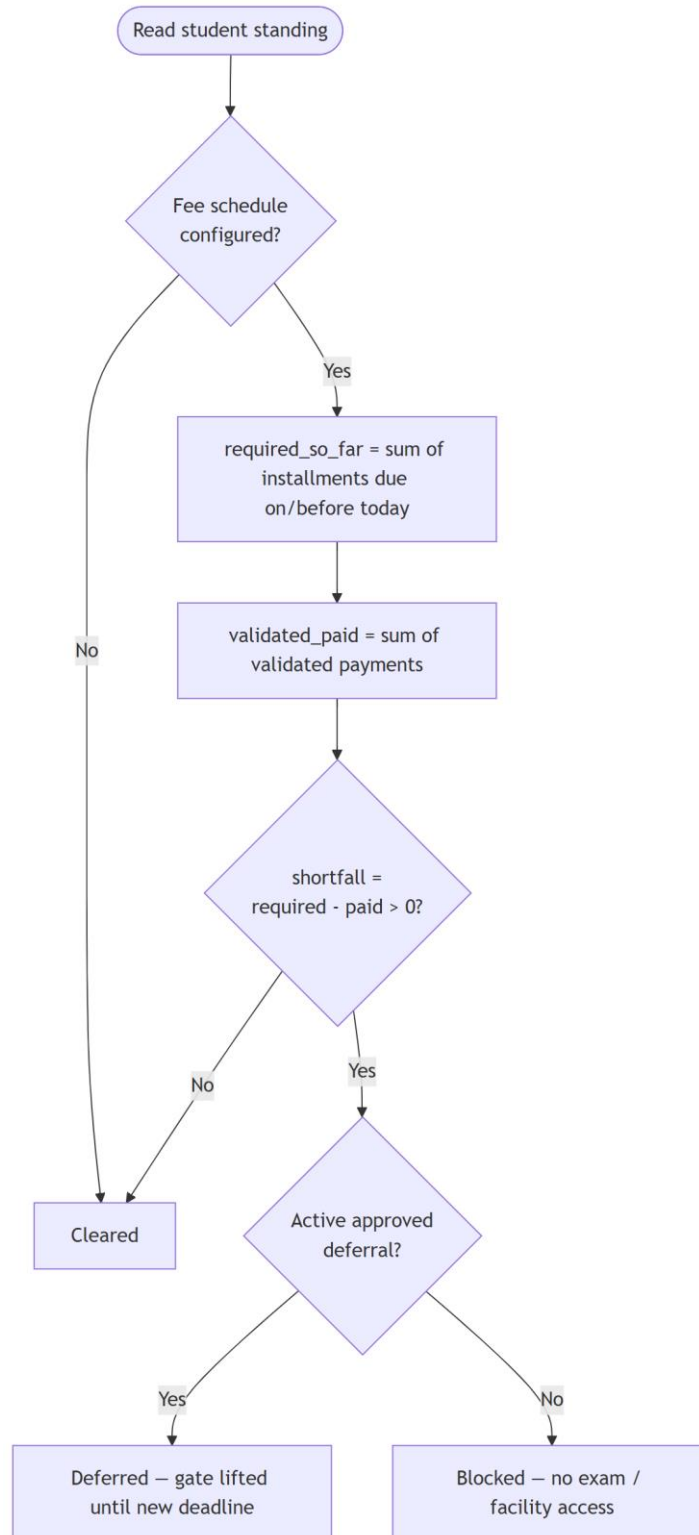


Figure 8: Activity diagram — payment standing and exam-gating

2.4.5 State Diagram

Figure 9 shows the **application lifecycle** as a state machine: an application moves through reversible interim states under SAO triage before reaching one of the terminal decisions, of which only the admitted and withdrawn paths affect a student record.

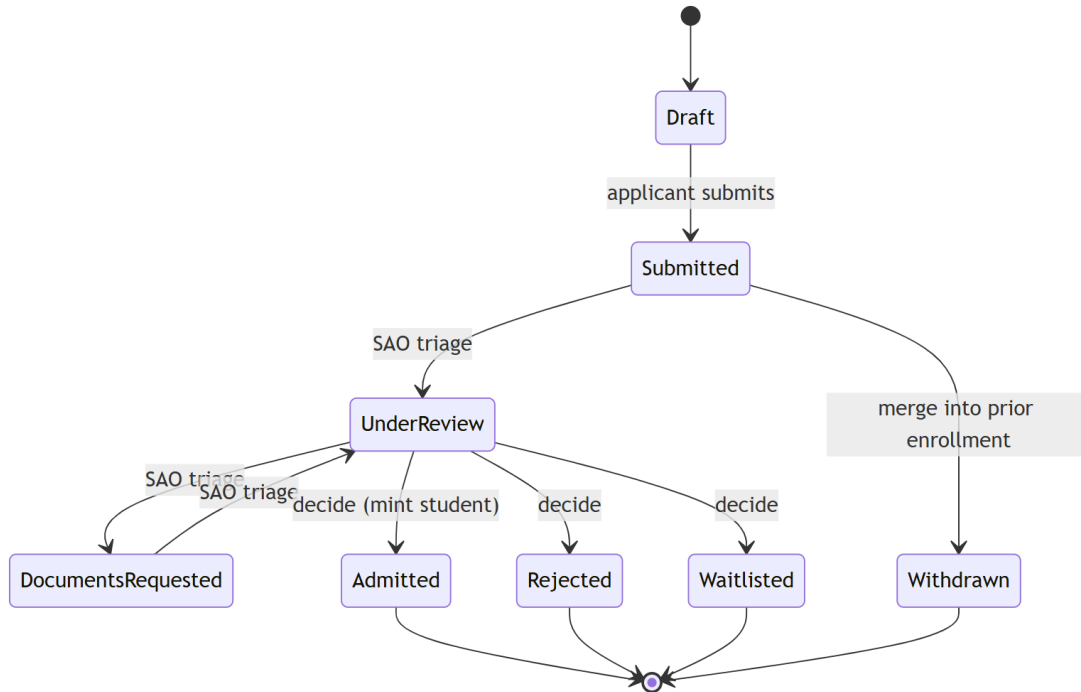


Figure 9: State diagram — application lifecycle

2.4.6 Database Design (Entity-Relationship Model)

The database comprises twenty-four domain tables plus supporting framework and counter tables. Status and type columns are stored as strings backed by PHP enumerations rather than native database enumerations, and almost every table carries soft-delete and audit capabilities. Figure 10 shows the entity-relationship diagram of the core tables, and Table 6 groups the principal entities by bounded context.

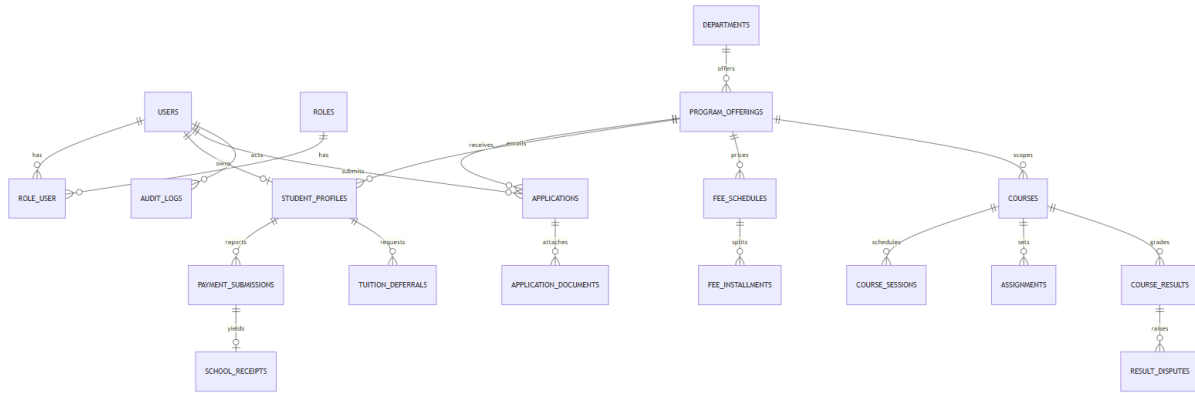


Figure 10: Entity-relationship diagram of the core database

Table 7: Principal database entities by bounded context

Context	Tables
Identity & roles	users, roles, role_user, student_profiles, lecturer_profiles, accountant_profiles, sao_profiles
Admissions	departments, program_offerings, document_types, level_credential_requirements, applications, application_documents
Payments & receipts	fee_schedules, fee_installments, payment_submissions, school_receipts
Exam-gating	tuition_deferrals (payment standing is derived, never stored)
Courses	courses, course_sessions, attendance_records, assignments, assignment_submissions, course_results, result_disputes
Notifications & audit	notifications, audit_logs

2.5 Needed Resources and Cost Estimation

The development of the system required human, hardware, and software resources. Because the project was built by a small student team on an open-source stack, the dominant cost is human effort; most software was free of charge.

2.5.1 Needed Resources

Human resources — a system analyst to define requirements, a UI/UX designer to model the diagrams and interfaces, front-end and back-end developers to build the application, a tester to verify functionality and security, and a project manager to coordinate the work.

Hardware resources — a development laptop, an external drive for backups, and an internet connection for research and deployment, as detailed in Table 8.

Software resources — the Laravel framework, the Inertia/Vue front-end stack, MySQL, Visual Studio Code, a UML modelling tool, and Git for version control — all open-source or free.

2.5.2 Cost Estimation

The estimated cost of the project, in CFA francs (XAF), is broken down into human, hardware, and software resources in Tables 7 to 10.

Table 8: Cost of human resources

Designation	Rate/day (XAF)	Days	Total (XAF)
Project manager	3 000	7	21 000
System analyst / designer	3 000	10	30 000
UI/UX designer	2 500	8	20 000
Front-end developer	5 500	15	82 500
Back-end developer	6 500	18	117 000
Tester	3 500	10	35 000
Total 1			305 500

Table 9: Cost of hardware resources

Designation	Unit price (XAF)	Quantity	Total (XAF)
Development laptop	400 000	1	400 000
External hard drive (1 TB)	35 000	1	35 000
Modem + internet (project duration)	30 000	1	30 000
Total 2			465 000

Table 10: Cost of software resources

Designation	Price (XAF)	Notes
Laravel, Inertia, Vue, MySQL	0	Open-source
Visual Studio Code, Git	0	Free
UML modelling tool	0	Free / community edition
Domain name + hosting (1 year)	25 000	Optional, for deployment
Total 3		25 000

Table 11: Total cost estimation

DESIGN AND IMPLEMENTATION OF A STUDENT MANAGEMENT SYSTEM

Item	Amount (XAF)
Total 1 — human resources	305 500
Total 2 — hardware resources	465 000
Total 3 — software resources	25 000
Grand total	795 500

CHAPTER 3: RESULTS AND DISCUSSION

This chapter presents the practical outcomes of the SchuLyf Student Management System. It shows the principal application interfaces, discusses how each implemented feature meets the objectives defined in the previous chapters, and summarises the testing carried out to validate the system. The screenshots below were captured from the running application.

3.1 Application Interface Screenshots

3.1.1 Login Page

The login page authenticates every type of user through a single field that accepts an email, employee ID, phone number, or matricule, and redirects each user to the dashboard for their role.

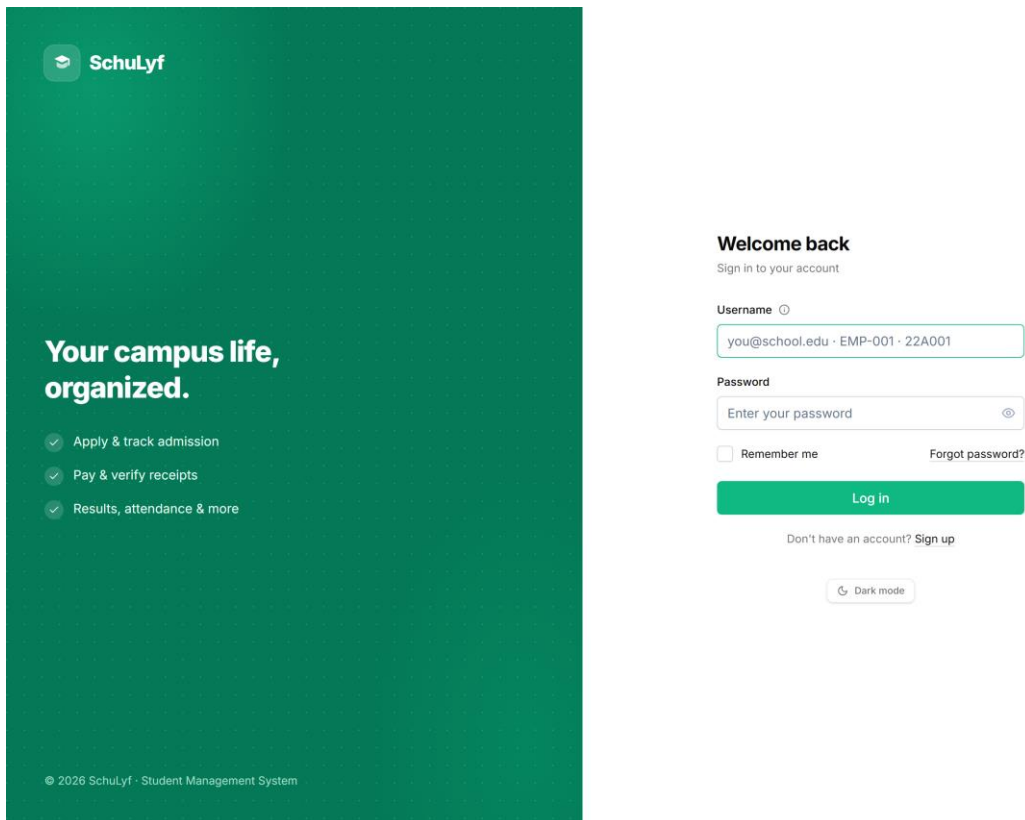


Figure 11: Login page

3.1.2 Applicant — Application Form

The application form guides an applicant through cascading programme and level menus and presents exactly the document slots required for the chosen programme, in addition to the always-required national ID and birth certificate.

Figure 12: Applicant application form

3.1.3 Student Affairs Officer — Review Queue

The SAO review screen lists applications in a filterable, paginated queue and allows the officer to triage an application through interim states and record a final admission decision.

STUDENT AFFAIRS Applications
Triage, review, and decide incoming applications.

Applicant	Programme	Level ↑↓	Status ↑↓	Submitted ↑↓	
Brian Tchoua #1	Computer Science Bachelors	1	Submitted	6/21/2026	Review
Clarisse Ngo #2	Computer Science Bachelors	1	Submitted	6/20/2026	Review
David Fometeu #3	Computer Science Bachelors	1	Under review	6/19/2026	Review
Esther Bih #4	Computer Science Bachelors	1	Documents requested	6/18/2026	Review

« < 1 > » 15 ▾

Figure 13: SAO application review queue

3.1.4 Accountant — Payment Review

The accountant reviews a reported deposit, views the uploaded bank slip inline, and validates it — issuing a tamper-proof school receipt — or rejects it with a reason.

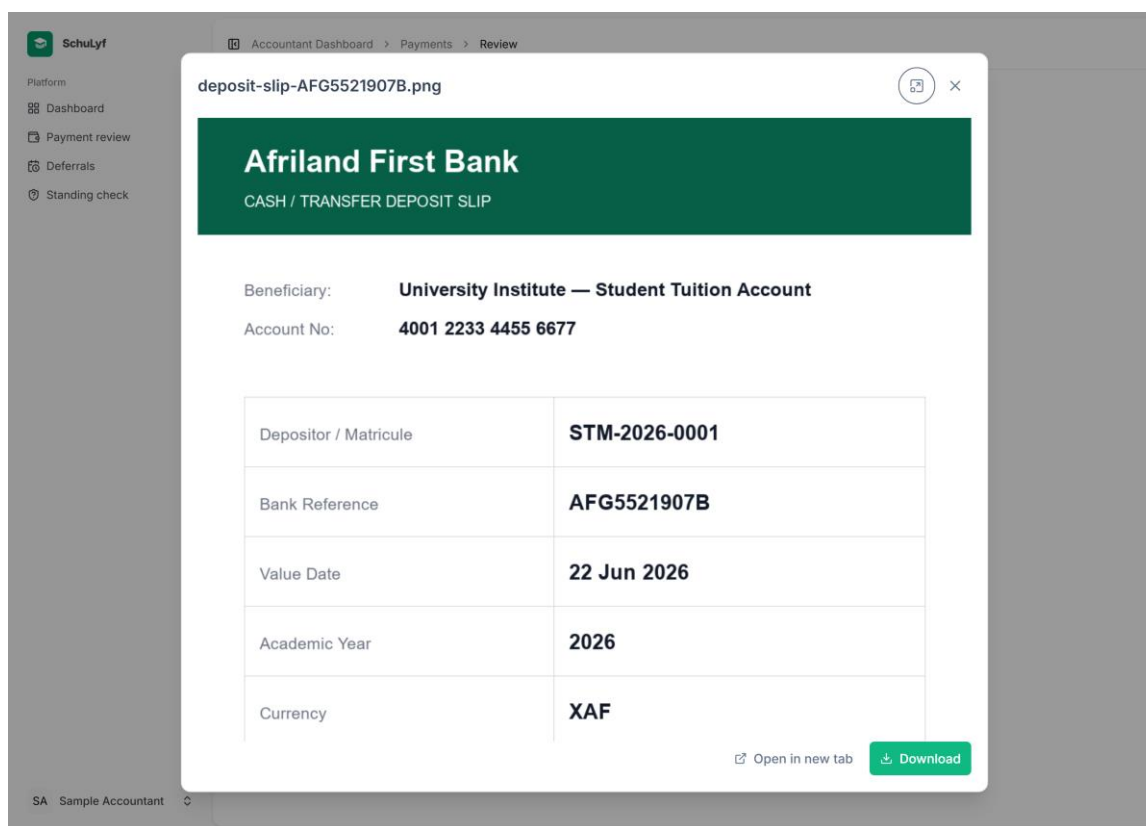


Figure 14: Accountant payment review with the inline bank-slip viewer

3.1.5 Student — Payments and Printable Receipt

The student payments screen shows the fee schedule, the validated-paid total, and the live payment standing, alongside a printable, QR-bearing school receipt that links to the public verification page.

SchulYf

Platform

Dashboard

My payments

My attendance

My assignments

My results

My payments

STUDENT

Tuition payments

Report deposits, track validation, and request a deferral.

Matricule

stm-2026-0001

Programme

Computer Science · Bachelors · L1

Academic year

2026

Cleared

You are cleared for exams and facilities.

Tuition total

500 000 FCFA

Validated paid

300 000 FCFA

Remaining

200 000 FCFA

Report a deposit

Bank

Select the bank

Amount (XAF)

Bank transaction reference

Bank slip

Choose file

No file chosen

Report payment

My reported deposits

Bank	Amount	Status	Reported	Slip	Receipt	Note
AFG AFG5521907B	200 000 FCFA	Submitted	6/22/2026	📄	—	—
UBA UBA8842130A	300 000 FCFA	Validated	6/22/2026	📄	📄 View	—

JN Jordan Nkeng

Figure 15: Student payments screen and payment standing

[← Back](#)[Print](#)

Official school receipt
Proof of validated tuition payment

Receipt no.
RCP-2026-00001

Student Jordan Nkeng	Matricule stm-2026-0001
Programme Computer Science - L1	Academic year 2026
Bank UBA (UBA8842130A)	Issued 6/22/2026

AMOUNT VALIDATED
300 000 FCFA

 **Tamper-proof**
Scan the code or visit the link to confirm this receipt is authentic and bound to the student above.
<https://student-management-system.test/receipts/verify/RCP-2026-00001>



Figure 16: Printable, QR-bearing school receipt

3.1.6 Public Receipt Verification

Anyone can verify a receipt number at a public page; an authentic receipt reveals the bound student identity and amount, while a forged or reused receipt is reported as invalid.

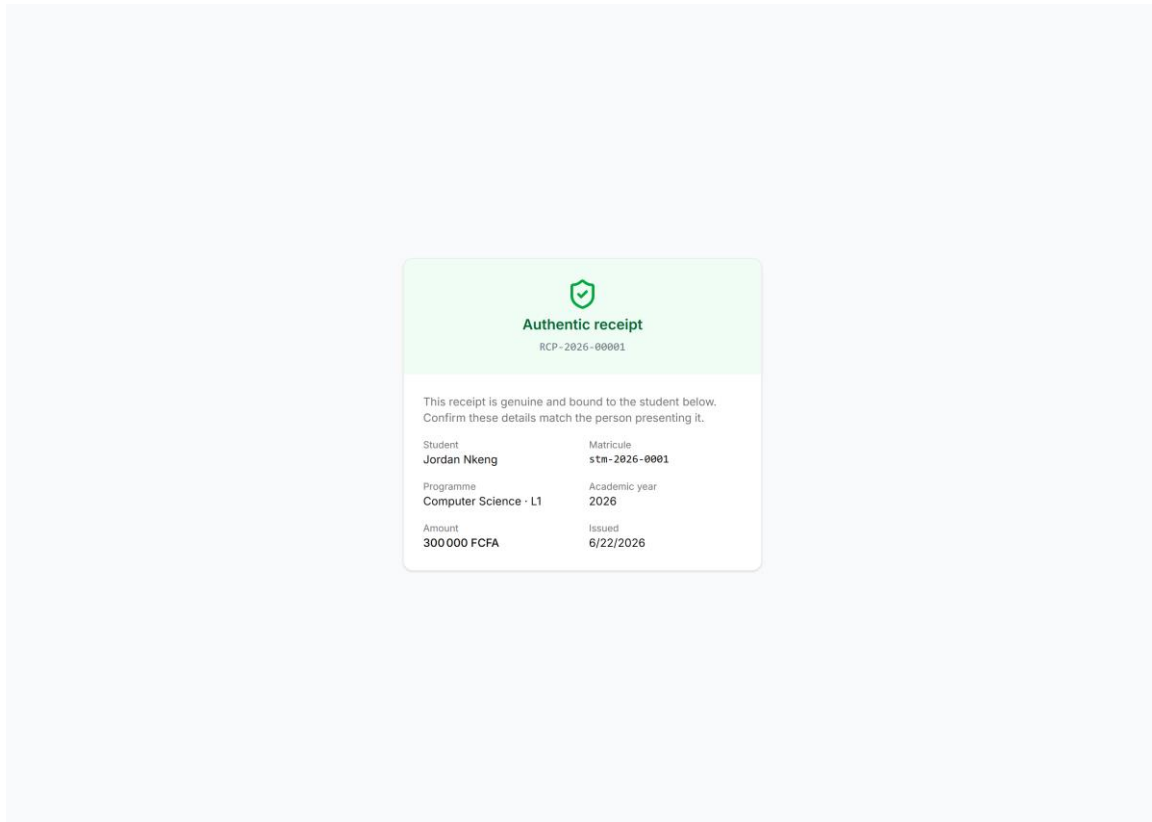


Figure 17: Public receipt verification page

3.1.7 Administrator — Dashboard and Audit Log

The administrator dashboard summarises users by role and applications by status and opens the immutable audit-log modal, where every significant action can be filtered by actor, action, subject, and date.

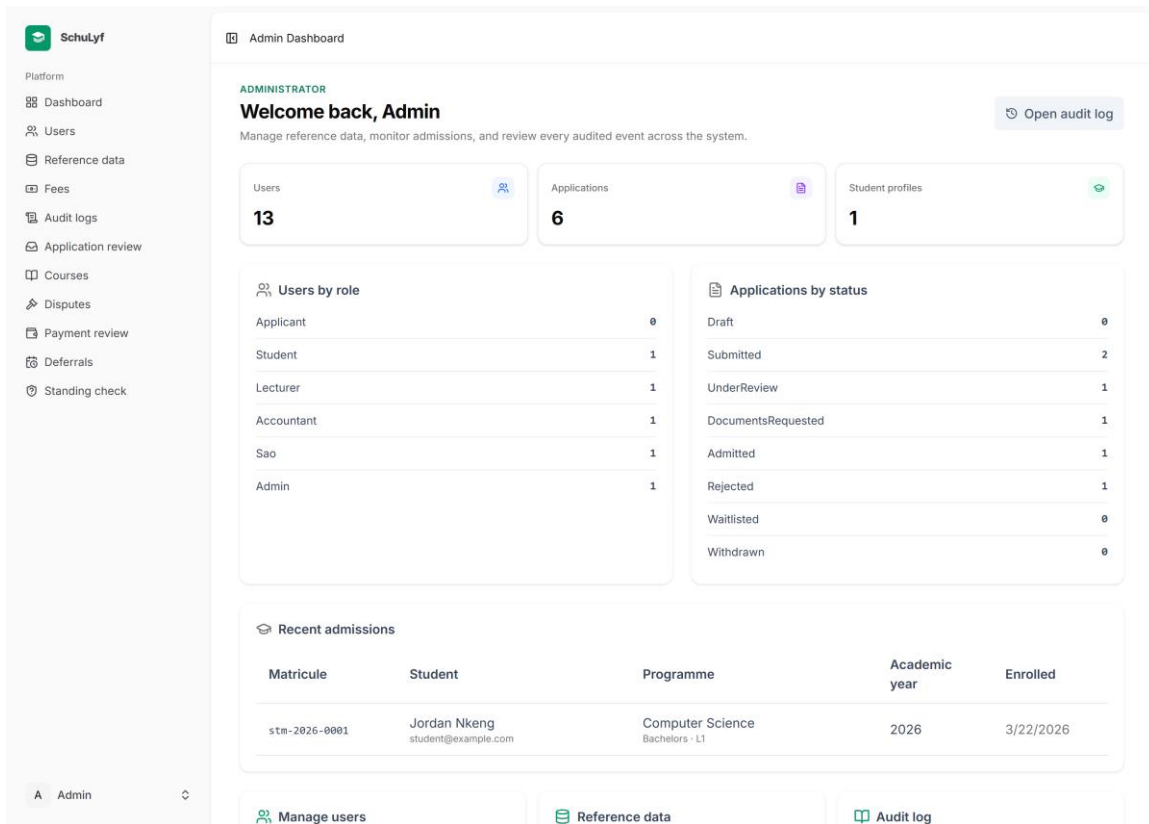


Figure 18: Administrator dashboard

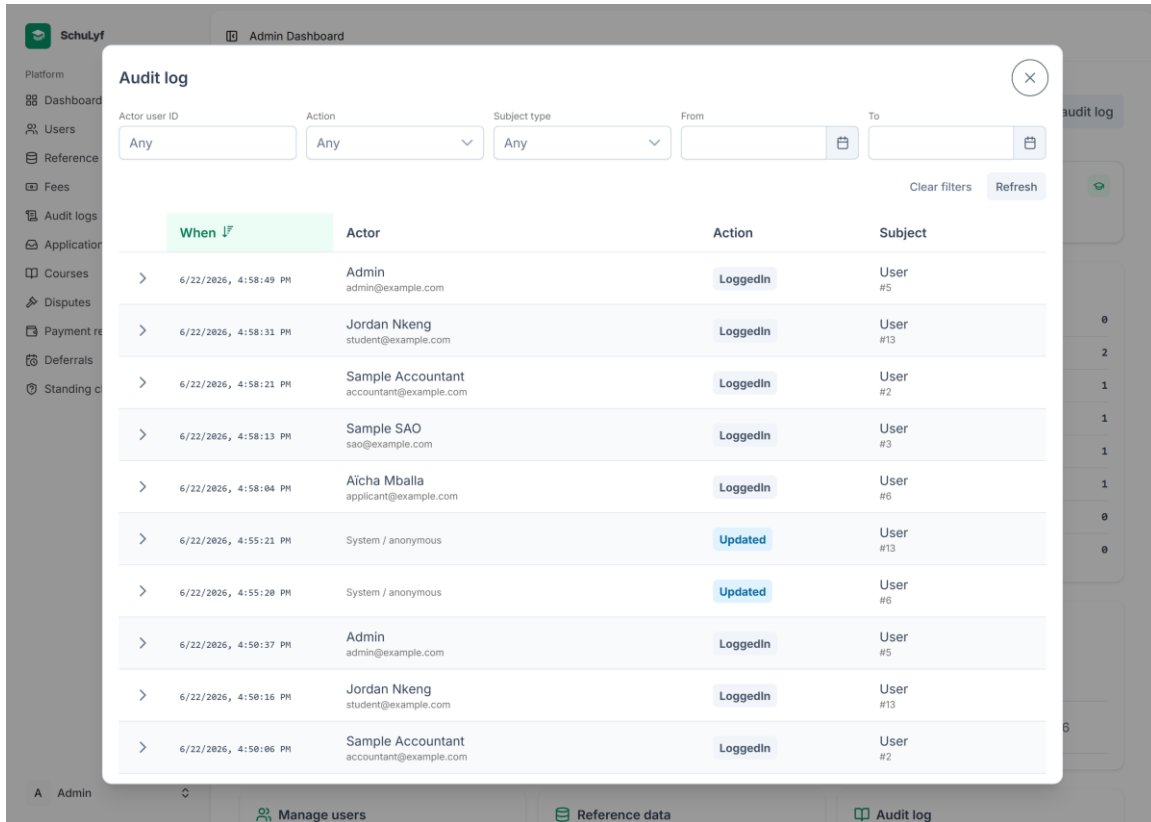


Figure 19: Immutable audit-log modal with filters

3.2 Testing and Validation

The system was validated with an extensive automated test suite written with the Pest framework, comprising several hundred feature and unit tests in addition to browser smoke tests. The tests cover authentication and authorization for every role, the admission decision and student-promotion flow, payment validation and receipt signature verification, the payment-standing and deferral logic, the full course-management lifecycle, and the immutability of the audit log and receipts. Code style is enforced automatically with Laravel Pint, and the front end is type-checked and linted. This automated coverage gives confidence that the implemented behaviour matches the specified requirements and that future changes will not silently break existing functionality.

3.3 Discussion

The implemented system meets the objectives set out in Chapter 1. Online admissions replace the paper-folder process and produce a unique matricule automatically. The tamper-proof,

HMAC-signed receipt with public verification resolves the twin problems of lost and forged paper receipts that motivated the project. Live payment standing and accountant-granted deferrals make exam-gating consistent and transparent. Course management brings planning, attendance, assignments, results, and disputes into one auditable workflow, and reliable cohort notifications replace distrusted word-of-mouth absence notices. Above all, the immutable audit log gives the institution a tamper-evident record of every significant action, directly addressing the accountability gaps of the manual system.

GENERAL CONCLUSION

The Student Management System developed in this project successfully addresses the major challenges of the traditional, manual student-administration processes of a Cameroonian university — processes that are fragmented, paper-bound, and prone to loss, forgery, and dispute. By digitising and automating admissions, payment validation, exam-gating, course management, and communication, SchuLyf significantly improves efficiency, transparency, and accountability.

The application provides a secure, role-based platform for six categories of user. Applicants apply and track decisions online; students report payments, monitor their standing, follow their courses, and verify their receipts; lecturers run their courses end to end; accountants validate payments and issue tamper-proof receipts; student-affairs officers decide admissions and oversee the academic workflow; and administrators manage the platform and audit it. The use of a robust relational database, HMAC-signed receipts, and an immutable audit log ensures reliable, tamper-evident record-keeping throughout.

Overall, the project achieved its objectives by delivering a functional, scalable, and well-tested Student Management System tailored to local needs. It demonstrates how a carefully designed, security-conscious web application can transform student administration. Future work could extend the platform with online payment-gateway integration, a dedicated mobile application, timetable generation, and richer analytics dashboards — all of which the layered architecture is designed to accommodate without redesign.

REFERENCES

- [1] Laravel. “Laravel 13 Documentation.” <https://laravel.com/docs> (accessed 2026).
- [2] Inertia.js. “The Modern Monolith.” <https://inertiajs.com> (accessed 2026).
- [3] Vue.js. “Vue 3 Documentation.” <https://vuejs.org> (accessed 2026).
- [4] PrimeVue. “Vue UI Component Library.” <https://primevue.org> (accessed 2026).
- [5] Tailwind CSS. “Documentation.” <https://tailwindcss.com> (accessed 2026).
- [6] MySQL. “MySQL 8.0 Reference Manual.” <https://dev.mysql.com/doc> (accessed 2026).
- [7] OWASP Foundation. “OWASP Top Ten.” <https://owasp.org/www-project-top-ten> (accessed 2026).
- [8] H. Krawczyk, M. Bellare, R. Canetti. “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, IETF, 1997.
- [9] Pest. “An Elegant PHP Testing Framework.” <https://pestphp.com> (accessed 2026).
- [10] OMG. “Unified Modeling Language (UML) Specification, v2.5.1,” Object Management Group, 2017.